Copy Constructor vs Assignment Operator and Rule of Three/Five/Zero

Theory, Edge Cases & Code Examples

- ✓ Fundamental Distinction
- ✓ Resource Management Guidelines
- ✓ Safety, Performance, and Correctness
- ✓ Self-Assignment Safety
- ✓ Copy Elision and Mandatory Optimization

→ Swipe to learn →

Copy Constructor vs Assignment Operator and Rule of Three/Five/Zero

Theory

📌 1. Copy Constructor vs Copy Assignment Operator - Fundamental Distinction (1/2)

When each is called:

Operation	Which Function	Object State	Syntax Example
Initialization from existing object	Copy Constructor	New object being created	`MyClass b = a;` or `MyClass b(a);`
Pass by value to function	Copy Constructor	New parameter object created	`void func(MyClass obj)`
Return by value from function	Copy Constructor (or elided)	New object in caller's scope	`return localObj;`
Assignment to existing object	Copy Assignment	Existing object being modified	`a = b;` (both already exist)
Chained assignment	Copy Assignment	Multiple existing objects	`a = b = c;`

Key differences table:

Aspect	Copy Constructor	Copy Assignment Operator
Signature	`T(const T& other)`	`T& operator=(const T& other)`
Purpose	Initialize NEW object from existing	Modify EXISTING object to copy another
Previous state	No previous state (object being created)	Must clean up old resources first
Return type	None (constructor)	`T&` (reference to *this for chaining)
Self-assignment check	Not needed	**Required** to prevent corruption
Initializer list	Can use	Cannot use

Copy Constructor vs Assignment Operator and Rule of Three/Five/Zero

Theory

📌 1. Copy Constructor vs Copy Assignment Operator - Fundamental Distinction (2/2)

Compiler-generated behavior:

Member Type	Default Copy Constructor	Default Copy Assignment
Built-in types (int, pointers)	Bitwise copy (shallow)	Bitwise copy (shallow)
Class types	Calls member's copy constructor	Calls member's copy assignment
Pointers	❌ **Shallow copy (dangerous!)**	❌ **Shallow copy (dangerous!)**
Smart pointers	Defined by smart pointer type	Defined by smart pointer type

CRITICAL: Default shallow copy of raw pointers causes **double-deletion** - multiple objects delete the same memory, causing crashes.

Code example:

```
class Data {
    int* ptr;
public:
    // ✅ Copy Constructor - creates new object
    Data(const Data& other) : ptr(new int(*other.ptr)) {
        // No previous state to clean up
    }

    // ✅ Copy Assignment - modifies existing object
    Data& operator=(const Data& other) {
        if (this != &other) {  // Self-assignment check required
            delete ptr;  // Clean up old resource first
            ptr = new int(*other.ptr);
        }
        return *this;  // Return *this for chaining
    }
};
```

Copy Constructor vs Assignment Operator and Rule of Three/Five/Zero

Theory

📌 2. Rule of Three, Five, and Zero - Resource Management Guidelines (1/2)

The three rules compared:

Rule	Era	When to Apply	Special Members to Define	Modern Recommendation
Rule of Zero	C++11+	Class doesn't manage resources	**None** - use smart pointers	✅ **Preferred approach**
Rule of Three	C++98	Class manages resources	Destructor + Copy Ctor + Copy Assign	Use only if legacy/C interface
Rule of Five	C++11+	Class manages resources	Rule of Three + Move Ctor + Move Assign	Use when Rule of Zero impossible

Rule of Five - all special member functions:

Function	Signature	Purpose	Mark noexcept?
Destructor	`~T()`	Release resources	Yes (implicit)
Copy Constructor	`T(const T&)`	Deep copy resources	No (may allocate)
Copy Assignment	`T& operator=(const T&)`	Copy with cleanup	No (may allocate)
Move Constructor	`T(T&&)`	Transfer ownership	**Yes (critical for performance)**
Move Assignment	`T& operator=(T&&)`	Transfer with cleanup	**Yes (critical for performance)**

Copy Constructor vs Assignment Operator and Rule of Three/Five/Zero

Theory

📌 2. Rule of Three, Five, and Zero - Resource Management Guidelines (2/2)

When each rule applies:

Scenario	Which Rule	Example
Using `std::unique_ptr`, `std::vector`, `std::string`	✅ Rule of Zero	`std::unique_ptr<int[]> data;` - no manual management
Managing raw pointers (C interface)	Rule of Five	`int* data = new int[100];` - must define all five
Legacy C++98 code	Rule of Three	No move semantics available
Implementing new RAII wrapper	Rule of Five	Creating custom smart pointer

Code example - Rule of Zero (MODERN APPROACH):

```
class ModernBuffer {
    std::unique_ptr<int[]> data;  // ✅ Automatic resource management
    size_t size;
public:
    ModernBuffer(size_t n) : data(std::make_unique<int[]>(n)), size(n) {}
    // ✅ No special member functions defined
    // unique_ptr provides correct move-only semantics automatically
};

// Usage:
ModernBuffer buf1(100);
// ModernBuffer buf2 = buf1;  // ❌ Error: not copyable (correct!)
ModernBuffer buf3 = std::move(buf1);  // ✅ Movable
```

Copy Constructor vs Assignment Operator and Rule of Three/Five/Zero

Theory

📌 3. Why Copy Semantics Matter - Safety, Performance, and Correctness (1/3)

Common bugs from incorrect copy handling:

Bug Type	Cause	Symptom	Fix
Double-deletion crash	Shallow copy of pointers	Crash when second object destructs	Implement deep copy or use smart pointers
Memory leak	No cleanup in assignment	Growing memory usage	Implement proper assignment with cleanup
Data corruption	Self-assignment without check	Object corrupts itself	Add `if (this != &other)` check
Use-after-free	Dangling pointer after move	Crash or undefined behavior	Nullify pointers after move
Performance degradation	Unnecessary deep copies	Slow performance with large objects	Implement move semantics

Performance impact table:

Operation	With Copy Only	With Move Semantics	Speedup
Returning large vector from function	O(n) copy	O(1) pointer swap	100x-1000x faster
Inserting into std::vector (reallocation)	O(n) copies	O(n) moves	10x-100x faster
Swapping two large objects	3 deep copies	3 pointer swaps	100x-1000x faster
Passing temporary to function	Copy temporary	Move temporary	No allocation

📌 3. Why Copy Semantics Matter - Safety, Performance, and Correctness (2/3)

Self-assignment safety:

Scenario	Self-Assignment Possible?	Why?
`obj = obj;`	Yes (obviously)	Direct self-assignment
`*ptr1 = *ptr2;`	Yes (if ptr1 == ptr2)	Aliased pointers
`arr[i] = arr[j];`	Yes (if i == j)	Array index aliasing
`container[key1] = container[key2];`	Yes (if key1 == key2)	Container aliasing
Generic algorithms	Yes (algorithm doesn't know)	`std::copy`, `std::swap`, etc.

Copy Constructor vs Assignment Operator and Rule of Three/Five/Zero

Theory

📌 3. Why Copy Semantics Matter - Safety, Performance, and Correctness (3/3)

Code example - self-assignment bug:

```
class Unsafe {
    int* data;
public:
    Unsafe& operator=(const Unsafe& other) {
        delete[] data;  // ❌ If this == &other, deleted own data!
        data = new int[100];
        std::copy(other.data, other.data + 100, data);  // ❌ Copying from deleted memory
        return *this;
    }
};

class Safe {
    int* data;
public:
    Safe& operator=(const Safe& other) {
        if (this != &other) {  // ✅ Self-assignment check
            delete[] data;
            data = new int[100];
            std::copy(other.data, other.data + 100, data);
        }
        return *this;
    }
};
```

Key takeaway: Follow Rule of Zero whenever possible using smart pointers and RAII. When managing raw resources, implement Rule of Five correctly with self-assignment checks, deep copying, and noexcept moves. Copy semantics bugs cause crashes, leaks, and corruption that are difficult to debug.

Copy Constructor vs Assignment Operator and Rule of Three/Five/Zero

Edge Cases

📌 1. Self-Assignment Safety

Self-assignment occurs when an object is assigned to itself: `obj = obj;`. While seemingly unlikely, it happens in generic code with aliases, references, and during algorithm operations. A naive copy assignment operator that deletes resources before copying can corrupt the object during self-assignment.

```
class Unsafe {
    int* data;
public:
    Unsafe& operator=(const Unsafe& other) {
        delete[] data;  // ❌ Dangerous if this == &other
        data = new int[100];
        std::copy(other.data, other.data + 100, data);  // ❌ Copying from deleted memory
        return *this;
    }
};

class Safe {
    int* data;
public:
    Safe& operator=(const Safe& other) {
        if (this != &other) {  // ✅ Self-assignment check
            delete[] data;
            data = new int[100];
            std::copy(other.data, other.data + 100, data);
        }
        return *this;
    }
};
```

The standard idiom is to check `if (this != &other)` at the start of the assignment operator. Alternatively, use the copy-and-swap idiom, which is naturally self-assignment safe: create a temporary copy, then swap contents with it. The temporary's destructor cleans up the old resources. This technique also provides strong exception safety—if copying fails, the original object remains unchanged.

Copy Constructor vs Assignment Operator and Rule of Three/Five/Zero

Edge Cases

📌 2. Copy Elision and Mandatory Optimization

Copy elision is a compiler optimization that eliminates copy/move operations, directly constructing objects in their final destination. Before C++17, it was optional; compilers could elide copies but weren't required to. C++17 made copy elision **mandatory** in certain cases, fundamentally changing the language semantics.

```
class NoCopy {
public:
    NoCopy() = default;
    NoCopy(const NoCopy&) = delete;  // ❌ Copy deleted
    NoCopy(NoCopy&&) = delete;        // ❌ Move deleted
};

NoCopy factory() {
    return NoCopy();  // ✅ C++17: guaranteed elision, compiles
}

int main() {
    NoCopy obj = factory();  // ✅ No copy/move needed in C++17
}
```

Pre-C++17, this code wouldn't compile because the copy/move constructor must be accessible even if elided. C++17's guaranteed copy elision means the object is constructed directly in the caller's storage, so copy/move constructors don't need to exist. This changes observable behavior—copy/move constructor side effects no longer occur. Understanding when elision is guaranteed versus optional is crucial for reasoning about object lifetimes and performance.

Copy Constructor vs Assignment Operator and Rule of Three/Five/Zero

Edge Cases

📌 3. Implicit Generation Suppression Rules

The compiler generates special member functions under complex interdependent rules. Defining one function can prevent generation of others, and these rules changed between C++98 and C++11, creating subtle version-dependent behavior.

```
class Suppressed {
public:
    Suppressed(const Suppressed&) { }  // User-defined copy constructor
    // ❌ Move constructor NOT generated
    // ❌ Move assignment NOT generated
    // ✅ Copy assignment still generated (deprecated behavior)
};

class Modern {
public:
    Modern(Modern&&) = default;  // User-defined move constructor
    // ❌ Copy constructor NOT generated
    // ❌ Copy assignment NOT generated
    // ✅ Move assignment may be generated
};
```

Defining a copy constructor or copy assignment suppresses move operations. Defining move operations suppresses copy operations. Defining a destructor doesn't suppress copy operations in C++98 (for backward compatibility) but should, according to modern guidelines. This is why the Rule of Five exists—to explicitly control all special member functions rather than relying on complex implicit rules that vary by C++ version and compiler.

Copy Constructor vs Assignment Operator and Rule of Three/Five/Zero

Edge Cases

📌 4. Object Slicing in Copy Operations

Object slicing occurs when copying a derived class object to a base class object, discarding the derived portion. This destroys polymorphic behavior because only the base part is copied, and the vtable pointer points to the base class.

```
class Base {
public:
    int baseData;
    virtual void identify() { std::cout << "Base\n"; }
};

class Derived : public Base {
public:
    int derivedData;
    void identify() override { std::cout << "Derived\n"; }
};

void processBase(Base b) {  // ❌ Pass by value causes slicing
    b.identify();  // Always prints "Base"
}

int main() {
    Derived d;
    d.derivedData = 42;
    Base b = d;  // ❌ Slicing: derivedData lost, vtable changed
    b.identify();  // Prints "Base"

    processBase(d);  // ❌ Slicing in function parameter
}
```

Slicing is rarely intentional and usually indicates a design error. Prevent it by passing polymorphic objects by pointer or reference, making base class copy constructor protected or deleted, or using `std::reference_wrapper`. Slicing also occurs in containers— `std::vector<Base>` cannot hold Derived objects polymorphically; use `std::vector<std::unique_ptr<Base>>` instead.

Copy Constructor vs Assignment Operator and Rule of Three/Five/Zero

Edge Cases

📌 5. Return Value and Named Return Value Optimization

Return Value Optimization (RVO) and Named Return Value Optimization (NRVO) eliminate copy/move operations when returning objects. RVO applies to returning temporaries; NRVO applies to returning named local variables. Understanding when these optimizations apply helps write efficient code.

```
Object createRVO() {
    return Object();  // ✅ RVO: definitely elided in C++17
}

Object createNRVO() {
    Object local;
    // ... use local
    return local;  // ⚠️ NRVO: might be elided (not guaranteed)
}

Object ambiguousNRVO(bool condition) {
    Object a, b;
    return condition ? a : b;  // ❌ NRVO cannot apply (ambiguous)
}
```

RVO is guaranteed in C++17 for returning temporaries. NRVO is never guaranteed—if the compiler can't determine which object is returned at compile time, it cannot apply NRVO. Multiple return paths or conditional returns prevent NRVO. Using `std::move` on return values can prevent NRVO while forcing a move, potentially pessimizing performance. The best practice is to return local objects directly without `std::move` and let the compiler optimize.

Copy Constructor vs Assignment Operator and Rule of Three/Five/Zero

Edge Cases

📌 6. Deleted Special Members and Copy Elision Interaction

Deleting copy/move constructors prevents explicit copying/moving, but doesn't prevent all object creation due to mandatory copy elision in C++17. This creates counterintuitive situations where objects with deleted constructors can still be returned and initialized.

```
class Uncopyable {
public:
    Uncopyable() = default;
    Uncopyable(const Uncopyable&) = delete;
    Uncopyable(Uncopyable&&) = delete;
};

Uncopyable factory() {
    return Uncopyable();  // ✅ C++17: elision, no copy/move needed
}

Uncopyable createLocal() {
    Uncopyable local;
    return local;  // ❌ Error: NRVO not guaranteed, move needed but deleted
}

int main() {
    Uncopyable obj = factory();  // ✅ Works in C++17
}
```

Returning a temporary with deleted copy/move works due to mandatory elision. Returning a named local variable fails because NRVO isn't guaranteed, and the fallback move is deleted. This behavior is useful for types that should only be constructed in-place, like `std::atomic` or unique resource handles. However, it limits usage patterns—you cannot store them in containers or pass them through interfaces that require copying/moving.

Copy Constructor vs Assignment Operator and Rule of Three/Five/Zero

Code Examples

📌 1. Basic Copy Constructor and Assignment Operator

This demonstrates the fundamental pattern: copy constructor allocates new resources and copies data, while assignment operator must first clean up existing resources before allocating and copying. The self-assignment check prevents disaster if an object is assigned to itself.

```
class SimpleString {
    char* data;
    size_t length;

public:
    SimpleString(const char* str) {
        length = strlen(str);
        data = new char[length + 1];
        strcpy(data, str);
    }

    // ✅ Copy constructor - creates new object
    SimpleString(const SimpleString& other)
        : length(other.length), data(new char[other.length + 1]) {
        strcpy(data, other.data);
        std::cout << "Copy constructor\n";
    }

    // ✅ Copy assignment operator - modifies existing object
    SimpleString& operator=(const SimpleString& other) {
        if (this != &other) {  // Self-assignment check
            delete[] data;  // Clean up old resource
            length = other.length;
            data = new char[length + 1];
            strcpy(data, other.data);
        }
        std::cout << "Copy assignment\n";
        return *this;
    }

    ~SimpleString() {
        delete[] data;
    }
};
```

Copy Constructor vs Assignment Operator and Rule of Three/Five/Zero

Code Examples

📌 2. Rule of Three Implementation

The Rule of Three is fully implemented: destructor releases resources, copy constructor performs deep copy, and copy assignment handles cleanup before copying. All three work together to ensure proper resource management throughout the object's lifetime.

```
class DynamicArray {
    int* data;
    size_t size;

public:
    DynamicArray(size_t n) : size(n), data(new int[n]) {
        std::fill(data, data + size, 0);
    }

    // ✅ Copy constructor
    DynamicArray(const DynamicArray& other)
        : size(other.size), data(new int[other.size]) {
        std::copy(other.data, other.data + size, data);
    }

    // ✅ Copy assignment operator
    DynamicArray& operator=(const DynamicArray& other) {
        if (this != &other) {
            delete[] data;
            size = other.size;
            data = new int[size];
            std::copy(other.data, other.data + size, data);
        }
        return *this;
    }

    // ✅ Destructor
    ~DynamicArray() {
        delete[] data;
    }
};
```

Copy Constructor vs Assignment Operator and Rule of Three/Five/Zero

Code Examples

📌 3. Rule of Five with Move Semantics (1/2)

Move operations transfer ownership by stealing resources from the source object, which is left in a valid but empty state (nullified pointers). Move operations should be `noexcept` for optimal container performance—containers use move operations only if they're guaranteed not to throw.

```
class Buffer {
    char* data;
    size_t capacity;

public:
    Buffer(size_t n) : capacity(n), data(new char[n]) {}

    // Copy constructor
    Buffer(const Buffer& other)
        : capacity(other.capacity), data(new char[other.capacity]) {
        std::copy(other.data, other.data + capacity, data);
        std::cout << "Copy\n";
    }

    // Copy assignment
    Buffer& operator=(const Buffer& other) {
        if (this != &other) {
            delete[] data;
            capacity = other.capacity;
            data = new char[capacity];
            std::copy(other.data, other.data + capacity, data);
        }
        std::cout << "Copy assign\n";
        return *this;
    }

    // ✅ Move constructor
    Buffer(Buffer&& other) noexcept
        : capacity(other.capacity), data(other.data) {
        other.data = nullptr;  // ✅ Nullify source
        other.capacity = 0;
        std::cout << "Move\n";
    }

    // ✅ Move assignment
// ...continues on next slide
```

Copy Constructor vs Assignment Operator and Rule of Three/Five/Zero

Code Examples

📌 3. Rule of Five with Move Semantics (2/2)

[Code continues across slides]

```
// ...continued from previous slide
    Buffer& operator=(Buffer&& other) noexcept {
        if (this != &other) {
            delete[] data;  // Clean up current resource
            data = other.data;
            capacity = other.capacity;
            other.data = nullptr;  // ✅ Nullify source
            other.capacity = 0;
        }
        std::cout << "Move assign\n";
        return *this;
    }

    ~Buffer() {
        delete[] data;
    }
};
```

Copy Constructor vs Assignment Operator and Rule of Three/Five/Zero

Code Examples

📌 4. Copy-and-Swap Idiom

The copy-and-swap idiom provides strong exception safety and automatic self-assignment safety. The assignment operator takes its parameter by value (invoking copy constructor), then swaps contents. The temporary (containing the old data) is destroyed automatically, cleaning up resources. This elegant pattern combines assignment with the copy constructor's logic.

```
class SafeResource {
    int* data;
    size_t size;

public:
    SafeResource(size_t n) : size(n), data(new int[n]) {}

    SafeResource(const SafeResource& other)
        : size(other.size), data(new int[other.size]) {
        std::copy(other.data, other.data + size, data);
    }

    // ✅ Copy-and-swap idiom
    SafeResource& operator=(SafeResource other) {  // Pass by value (copy)
        swap(other);  // Swap with temporary
        return *this;  // Temporary destroyed, cleaning up old data
    }

    void swap(SafeResource& other) noexcept {
        std::swap(data, other.data);
        std::swap(size, other.size);
    }

    ~SafeResource() {
        delete[] data;
    }
};
```

Copy Constructor vs Assignment Operator and Rule of Three/Five/Zero

Code Examples

📌 5. Preventing Copying and Moving

Explicitly deleting copy and move operations prevents object duplication, useful for singletons, resource managers, or types representing unique system resources. This is preferable to the old C++98 technique of declaring them private without definition.

```
class Singleton {
public:
    static Singleton& getInstance() {
        static Singleton instance;
        return instance;
    }

    // ❌ Delete copy operations
    Singleton(const Singleton&) = delete;
    Singleton& operator=(const Singleton&) = delete;

    // ❌ Delete move operations
    Singleton(Singleton&&) = delete;
    Singleton& operator=(Singleton&&) = delete;

private:
    Singleton() = default;  // Private constructor
    ~Singleton() = default;
};
```

Copy Constructor vs Assignment Operator and Rule of Three/Five/Zero

Code Examples

📌 6. Rule of Zero with Smart Pointers

Following the Rule of Zero, this class defines no special member functions. The unique_ptr automatically manages memory, provides move semantics, and prevents copying. The class inherits these properties naturally. This is the modern C++ approach—use RAII wrappers instead of manual resource management.

```
class ModernBuffer {
    std::unique_ptr<int[]> data;  // ✅ Smart pointer manages memory
    size_t size;

public:
    ModernBuffer(size_t n) : data(std::make_unique<int[]>(n)), size(n) {}

    // ✅ No special member functions defined
    // unique_ptr is movable but not copyable
    // ModernBuffer inherits this behavior

    int& operator[](size_t i) { return data[i]; }
};

// Usage:
ModernBuffer buf1(100);
// ModernBuffer buf2 = buf1;  // ❌ Error: not copyable
ModernBuffer buf3 = std::move(buf1);  // ✅ OK: movable
```

Copy Constructor vs Assignment Operator and Rule of Three/Five/Zero

Code Examples

📌 7. Distinguishing Copy Constructor from Assignment

The key distinction: `=` during declaration is initialization (copy constructor), while `=` for an existing object is assignment. This syntax confusion catches many developers. Parentheses notation `Demo c(a)` makes the copy constructor explicit.

```
class Demo {
public:
    Demo() { std::cout << "Default\n"; }
    Demo(const Demo&) { std::cout << "Copy ctor\n"; }
    Demo& operator=(const Demo&) {
        std::cout << "Copy assign\n";
        return *this;
    }
};

int main() {
    Demo a;               // Default constructor
    Demo b = a;           // ✅ Copy constructor (initialization)
    Demo c(a);            // ✅ Copy constructor (explicit)

    Demo d;               // Default constructor
    d = a;                // ✅ Copy assignment (existing object)

    Demo e;
    e = Demo();           // Copy assignment (temporary)
}
```

Copy Constructor vs Assignment Operator and Rule of Three/Five/Zero

Code Examples

📌 8. Self-Assignment Without Check

The dangerous version fails on self-assignment because it deletes the resource before copying, corrupting itself. The safe version checks for self-assignment or allocates new resources before deleting old ones (strong exception safety). This demonstrates why self-assignment checks or copy-and-swap are essential.

```
class Dangerous {
    int* data;
public:
    Dangerous& operator=(const Dangerous& other) {
        delete[] data;  // ❌ Deletes resource
        data = new int[100];
        std::copy(other.data, other.data + 100, data);  // ❌ Copies from deleted memory
        return *this;
    }
};

class Safe {
    int* data;
public:
    Safe& operator=(const Safe& other) {
        if (this != &other) {  // ✅ Self-assignment check
            int* newData = new int[100];  // ✅ Allocate first
            std::copy(other.data, other.data + 100, newData);
            delete[] data;  // ✅ Delete old after copy succeeds
            data = newData;
        }
        return *this;
    }
};
```

What You Learned

Key Takeaways

- ✓ Fundamental Distinction
- ✓ Resource Management Guidelines
- ✓ Safety, Performance, and Correctness
- ✓ Self-Assignment Safety

→ *Swipe to learn* →